Code review: Week 1

PROT – Translating RNA into Protein

우상혁

문제

입력: mRNA 가닥에 해당하는 RNA 문자열, 길이는 최대 10 kbp

출력: 이 RNA가 암호화하는 단백질 문자열

샘플 입력 `AUGGCCAUGGCGCCCAGAACUGAGAUCAAUAGUACCCGUAUUAACGGGUGA`

샘플 출력 `MAMAPRTEINSTRING`

단백질 문자열은 B, J, O, U, X, Z를 뺀 알파벳 20글자로 표기

입력은 시작 코돈 AUG로 시작하고, 중간의 AUG는 M 하나로 번역됨

출처: Rosalind PROT 문제 설명과 샘플 데이터

번역

샘플 서열을 세 염기씩 끊어 코돈표에서 아미노산을 찾음

AUG	GCC	AUG	GCG	CCC	AGA	ACU	GAG	AUC
M	A	M	A	P	R	T	E	I

AAU	AGU	ACC	CGU	AUU	AAC	GGG	UGA
N	S	T	R	I	N	G	Stop

결과 `MAMAPRTEINSTRING`

출처: Rosalind PROT 샘플 데이터

코돈표

코돈은 64개이고, 61개가 아미노산 20종을 암호화함
3개(UAA, UAG, UGA)는 지정하는 아미노산이 없음

아미노산보다 코돈이 많아서 여러 코돈이 같은 아미노산을 가리킴
이것을 degeneracy라고 부름

	b=U		b=C		b=A		b=G		
U	UUU	F Phe	UCU	S Ser	UAU	Y Tyr	UGU	C Cys	U
	UUC	F Phe	UCC	S Ser	UAC	Y Tyr	UGC	C Cys	C
	UUA	L Leu	UCA	S Ser	UAA	* Stop	UGA	* Stop	A
	UUG	L Leu	UCG	S Ser	UAG	* Stop	UGG	W Trp	G
C	CUU	L Leu	CCU	P Pro	CAU	H His	CGU	R Arg	U
	CUC	L Leu	CCC	P Pro	CAC	H His	CGC	R Arg	C
	CUA	L Leu	CCA	P Pro	CAA	Q Gln	CGA	R Arg	A
	CUG	L Leu	CCG	P Pro	CAG	Q Gln	CGG	R Arg	G
A	AUU	I Ile	ACU	T Thr	AAU	N Asn	AGU	S Ser	U
	AUC	I Ile	ACC	T Thr	AAC	N Asn	AGC	S Ser	C
	AUA	I Ile	ACA	T Thr	AAA	K Lys	AGA	R Arg	A
	AUG	M Met	ACG	T Thr	AAG	K Lys	AGG	R Arg	G
G	GUU	V Val	GCU	A Ala	GAU	D Asp	GGU	G Gly	U
	GUC	V Val	GCC	A Ala	GAC	D Asp	GGC	G Gly	C
	GUA	V Val	GCA	A Ala	GAA	E Glu	GGA	G Gly	A
	GUG	V Val	GCG	A Ala	GAG	E Glu	GGG	G Gly	G

단백질 AI CaLM

번역은 61개 코돈을 아미노산 20종으로 암호화하는 다대일 변환

codons	AUG	GCC	AUG	GCG	CCC	...	AUG	GC**U**	AUG	GCG	CCC	...	다름
prot_answer	M	A	M	A	P	...	M	A	M	A	P	...	같음

아미노산으로 학습한 단백질 언어모델은 두 서열을 구별하지 못함

코돈으로 학습한 CaLM은 종 판별, 녹는점 예측 등 일부 과제에서 기존 단백질 언어모델을 앞섬

출처: Outeiral, Deane. Nature Machine Intelligence 2024, DOI 10.1038/s42256-024-00791-0

풀이 개요

1	코돈표 준비	`{"UUU": "F", "UUC": "F", ..., "GGG": "G"}`
2	서열 읽기	`"AUGGCCAUGGCGCCCAGAACU...GGGUGA"`
3	세 글자씩 잘라 아미노산 찾기	`AUG → M  GCC → A  AUG → M  ...`
4	종결 코돈에서 멈추기	`...  GGG → G  UGA → 멈춤`

제 코드 리뷰 ① 코돈표를 dict로 변환

```python
#코돈 표 텍스트 파일을 따로 만들고 열어서 딕셔너리로 변환
with open("C:\\Users\\user\\Downloads\\COMPASS_practice\\RNA codon table.txt", encoding="utf-8") as c:
    raw_amino=c.read().split()    #['UUU', 'F', ..., 'GGG', 'G']

codon_table = raw_amino[0::2]     #['UUU', 'CUU', ..., 'AGG', 'GGG']
amino_table = raw_amino[1::2]     #['F', ..., 'G']

codon_dict=dict(zip(codon_table,amino_table))   #{'UUU': 'F', ..., 'GGG': 'G'}
```

[11] ✓ 0.0s

RNA codon table.txt

파일 편집 보기

UUU F	CUU L	AUU I	GUU V
UUC F	CUC L	AUC I	GUC V
UUA L	CUA L	AUA I	GUA V
UUG L	CUG L	AUG M	GUG V
UCU S	CCU P	ACU T	GCU A
UCC S	CCC P	ACC T	GCC A
UCA S	CCA P	ACA T	GCA A
UCG S	CCG P	ACG T	GCG A
UAU Y	CAU H	AAU N	GAU D
UAC Y	CAC H	AAC N	GAC D
UAA Stop	CAA Q	AAA K	GAA E
UAG Stop	CAG Q	AAG K	GAG E
UGU C	CGU R	AGU S	GGU G
UGC C	CGC R	AGC S	GGC G
UGA Stop	CGA R	AGA R	GGA G
UGG W	CGG R	AGG R	GGG G

코돈표를 txt 파일로 두고,
홀수 번째와 짝수 번째를 나눠 dict로 묶음

제 코드 리뷰 ② 세 글자씩 잘라 번역

```
#파일 받아서 읽기
with open("C:\\Users\\user\\Downloads\\COMPASS_practice\\rosalind_prot.txt", encoding="utf-8") as origin_text:
    mrna=origin_text.read().strip()     #AUGCCC...UACUUAA

#반복문 이용해서 mrna 3개씩 slicing
codons = [mrna[i:i+3] for i in range (0, len(mrna)-2, 3)]     #['AUG', 'CCC', ..., 'ACU', 'UAA']

#코돈에 대응되는 아미노산 딕셔너리에서 찾아 지정하기
prot_answer = [codon_dict.get(x) for x in codons]
print("".join(prot_answer[:prot_answer.index("Stop")]))    #MPNR...LSCSLVT
```

[12] ✓ 0.0s Python

```
MPNRFFPRRSRNLRVTEWLSVTDEVCNFHILFQPVKSYHAQKLDVCLGSLVLPLNGHCPLRTGPINGTVRGVTKARFCLAWIRRSVLWAVPTVPLNSWPSGITDSHTCNQGHHSLSTTYQSKAKLFGANRPCAVLSVLRRSGRRENTPINKS
```

샘플 데이터를 넣었을 때의 출력

MAMAPRTEINSTRING

다른 풀이1 : 코돈표를 반복문으로 생성

```python
# 1. 코돈표 만들기 (표준 유전 암호, UCAG 순서)
BASES = "UCAG"
AAS = "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRRIIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"
CODON_TABLE = {}
i = 0
for a in BASES:
    for b in BASES:
        for c in BASES:
            CODON_TABLE[a + b + c] = AAS[i]
            i += 1

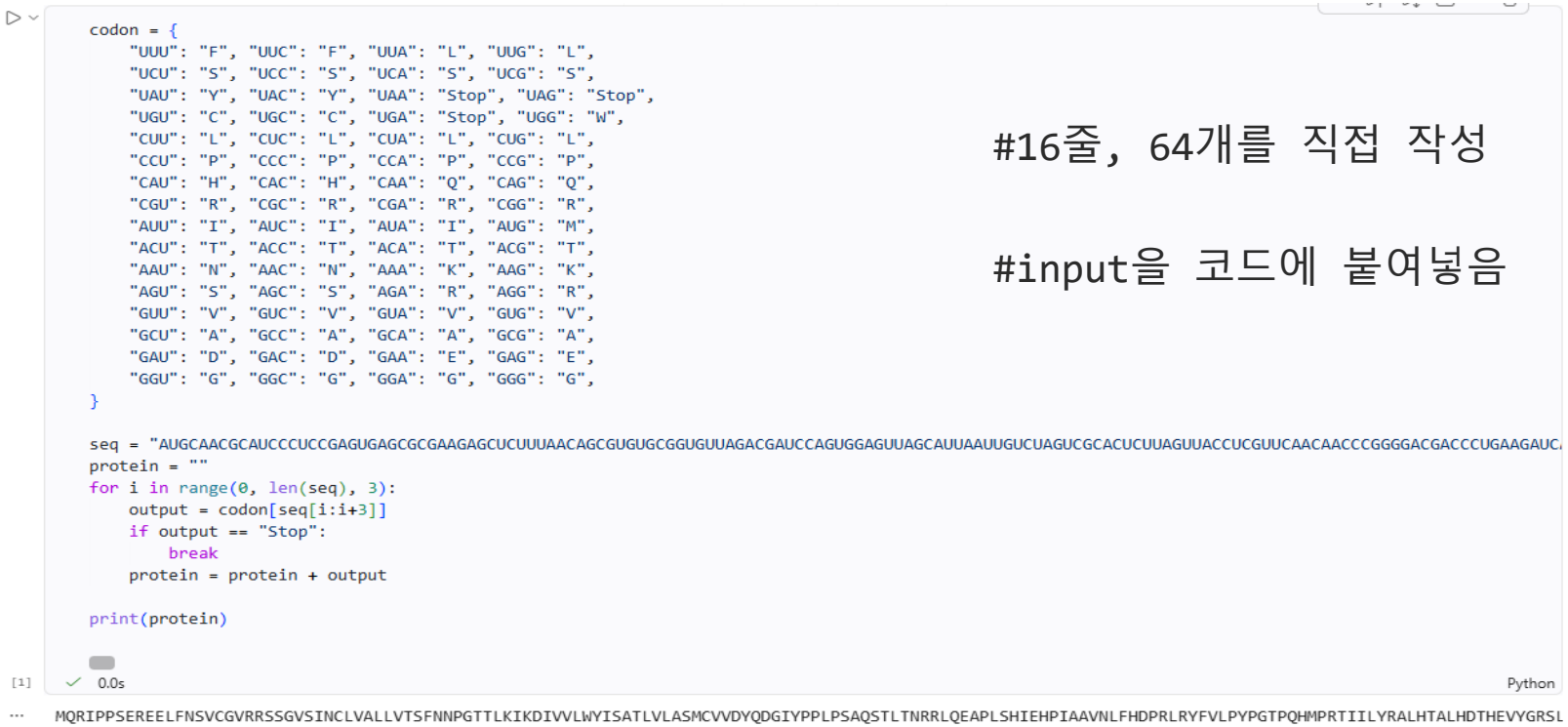
# 2. RNA -> 단백질 번역
def translate(rna):
    protein = []
    for j in range(0, len(rna) - 2, 3):
        aa = CODON_TABLE[rna[j:j+3]]
        if aa == "*":          # 종결 코돈(UAA, UAG, UGA)에서 멈춤
            break
        protein.append(aa)
    return "".join(protein)
```

UCAG를 3중으로 도는 순서와 AAS 글자 순서가 맞물려 64개가 짝지어짐

BASES를 ACGU로 바꾸면 오류 없이 AUG가 *인 틀린 표가 생김

https://claude.ai/artifact/XcjuR6ws4EQcP9HoatuXzY

다른 풀이2 : 코돈표와 서열을 직접 입력



```
codon = {
    "UUU": "F", "UUC": "F", "UUA": "L", "UUG": "L",
    "UCU": "S", "UCC": "S", "UCA": "S", "UCG": "S",
    "UAU": "Y", "UAC": "Y", "UAA": "Stop", "UAG": "Stop",
    "UGU": "C", "UGC": "C", "UGA": "Stop", "UGG": "W",
    "CUU": "L", "CUC": "L", "CUA": "L", "CUG": "L",
    "CCU": "P", "CCC": "P", "CCA": "P", "CCG": "P",
    "CAU": "H", "CAC": "H", "CAA": "Q", "CAG": "Q",
    "CGU": "R", "CGC": "R", "CGA": "R", "CGG": "R",
    "AUU": "I", "AUC": "I", "AUA": "I", "AUG": "M",
    "ACU": "T", "ACC": "T", "ACA": "T", "ACG": "T",
    "AAU": "N", "AAC": "N", "AAA": "K", "AAG": "K",
    "AGU": "S", "AGC": "S", "AGA": "R", "AGG": "R",
    "GUU": "V", "GUC": "V", "GUA": "V", "GUG": "V",
    "GCU": "A", "GCC": "A", "GCA": "A", "GCG": "A",
    "GAU": "D", "GAC": "D", "GAA": "E", "GAG": "E",
    "GGU": "G", "GGC": "G", "GGA": "G", "GGG": "G",
}

seq = "AUGCAACGCAUCCCUCCGAGUGAGCGCGAAGAGCUCUUUAACAGCGUGUGCGGUGUUAGACGAUCCAGUGGAGUUAGCAUUAAUUGUCUAGUCGCACUCUUAGUUACCUCGUUCAACAACCCGGGGACGACCCUGAAGAUC
protein = ""
for i in range(0, len(seq), 3):
    output = codon[seq[i:i+3]]
    if output == "Stop":
        break
    protein = protein + output

print(protein)
```

#16줄, 64개를 직접 작성

#input을 코드에 붙여넣음

[1] 0.0s Python

```
MQRIPPSEREELFNSVCGVRRSSGVSINCLVALLVTSFNNPGTTLKIKDIVVLWYISATLVLASMCVVDYQDGIYPPLPSAQSTLTNRRLQEAPLSHIEHPIAAVNLFHDPRLRYFVLPYPGTPQHMPRTIILYRALHTALHDTHEVYGRSL
```

64개를 직접 써서 가장 읽기 쉽고, 표준 코돈표와 전부 일치함

서열을 코드에 붙여넣어 새 데이터셋마다 코드를 고쳐야 함

풀이 비교

	나	다른풀이1	다른풀이2
코돈표	txt 파일을 읽어 dict(zip())	64글자 문자열과 UCAG 3중 반복문으로 생성	dict에 64개를 직접 작성
입력	파일 읽기, strip()	파일 읽기, 공백 제거, upper()	서열을 코드에 붙여넣음
종결 처리	끝까지 번역한 뒤 index("Stop")으로 자름	종결 코돈에서 break	종결 코돈에서 break
종결 코돈 없는 입력	ValueError 발생	끝까지 번역	끝까지 번역, 남은 염기에서 KeyError

Biopython이란

생물정보학용 파이썬 오픈소스 라이브러리

```
pip install biopython                          #터미널에서 설치하는 이름
from Bio.Seq import Seq                        #파이썬에서 import하는 이름
from Bio.Data import CodonTable

std = CodonTable.unambiguous_rna_by_id[1]      #NCBI 1번 표준 코돈표
std.forward_table["GCC"]                       #'A'
len(std.forward_table), std.stop_codons        #(61, ['UAA', 'UAG', 'UGA'])

dna = Seq("ATGGCCATGGCG")
dna.transcribe()                               #AUGGCCAUGGCG
dna.reverse_complement()                       #CGCCATGGCCAT
```

txt 코돈표 자리를 NCBI 번호별 코돈표가 대신함

출처: Cock 외. Bioinformatics 2009;25(11):1422-3, DOI 10.1093/bioinformatics/btp163

Biopython으로 풀면

```python
from Bio.Seq import Seq                          # Seq는 문자열처럼 쓰면서 전사·상보 같은 생물학 method
from Bio.Data import CodonTable                  # NCBI 유전암호표(번역표) 모음을 담은 모듈

std = CodonTable.unambiguous_rna_by_id[1]        # NCBI 번역표 1번(표준 유전암호) RNA 버전을 꺼냄. unambiguous =  A/C/G/U만, by_id = 번호로 찾는 딕셔너리

print(std.forward_table["GCC"])                  # forward_table은 {코돈: 아미노산 한 글자} 딕셔너리 GCC를 키로 넣으면 'A'

print(len(std.forward_table), std.stop_codons)   # forward_table은 아미노산 번역 코돈만 있음. 64 - 3(종결코돈) = 61개 stop_codons는 종결코돈 리스트

dna = Seq("ATGGCCATGGCG")                        # DNA 서열로 Seq 객체를 만든다. 이후 메서드들은 이 서열을 coding(sense) strand로 간주
print(dna.transcribe())                          # T를 U로 바꾸기만하고 입력을 coding strand로 보기 때문에 상보 변환은 하지않음 → AUGGCCAUGGCG
print(dna.reverse_complement())                  # A↔T, G↔C 상보 염기를 만들고 순서를 뒤집어, 반대쪽 가닥을 5'→3' 방향으로 돌려줌 → CGCCATGGCCAT
```

[4] ✓ 0.0s Python

```
A
61 ['UAA', 'UAG', 'UGA']
AUGGCCAUGGCG
CGCCATGGCCAT
```

to_stop=True를 주면 첫 종결 코돈 앞까지만 번역
주지 않으면 종결 코돈이 *로 찍힘

table=2는 척추동물 미토콘드리아 코돈표
-> UGA가 종결이 아니라 W

고칠 점

종결 코돈이 없는 서열을 넣으면
index("Stop")에서 ValueError 발생
break 방식과 Biopython의 to_stop=True는 끝까지 번역

코돈표 txt 파일이 repo에 없음

부록 ① 코드 문법 : 데이터 다루기

문법	뜻
`with open(파일) as f:`	파일을 열고, 블록이 끝나면 자동으로 닫음
`f.read()`	파일 내용 전체를 문자열 하나로 읽음
`s.strip()`	문자열 양끝의 공백과 줄바꿈을 지움
`s.split()`	공백과 줄바꿈 기준으로 잘라 리스트로 만듦
`s.upper()`	소문자를 대문자로 바꿈
`"".join(리스트)`	리스트 원소를 빈칸 없이 이어 붙임
`s[i:i+3]`	i번부터 세 글자를 잘라냄, 끝 번호는 미포함
`리스트[0::2]`	0번부터 두 칸씩 건너뛰며 뽑음
`dict(zip(a, b))`	두 리스트를 순서대로 짝지어 dict로 만듦
`{"UUU": "F", ...}`	dict를 직접 적어서 만듦
`d[key]`	없는 키면 KeyError 발생
`d.get(key)`	없는 키면 오류 대신 None 반환

부록 ② 코드 문법 : 흐름과 실행

문법	뜻
`range(시작, 끝, 간격)`	시작부터 끝 직전까지 간격만큼 커지는 수
`[식 for x in 반복]`	반복 결과를 바로 리스트로 만듦
`for 안의 for`	바깥 한 번 돌 때마다 안쪽이 전부 돎
`i += 1`	i = i + 1
`break`	가장 가까운 반복문을 바로 빠져나감
`lst.append(x)`	리스트 끝에 x를 추가
`s = s + x`	문자열을 새로 이어 붙여 다시 저장
`lst.index(x)`	x가 처음 나오는 위치, 없으면 ValueError
`def 함수(인자): ... return`	함수를 정의하고 결과를 돌려줌
`A if 조건 else B`	조건이 참이면 A, 아니면 B
`sys.argv`	터미널에서 실행할 때 적은 단어 리스트
`if __name__ == "__main__":`	파일을 직접 실행할 때만 아래 블록 실행